BRAIn lab
basic research of artificial intelligence

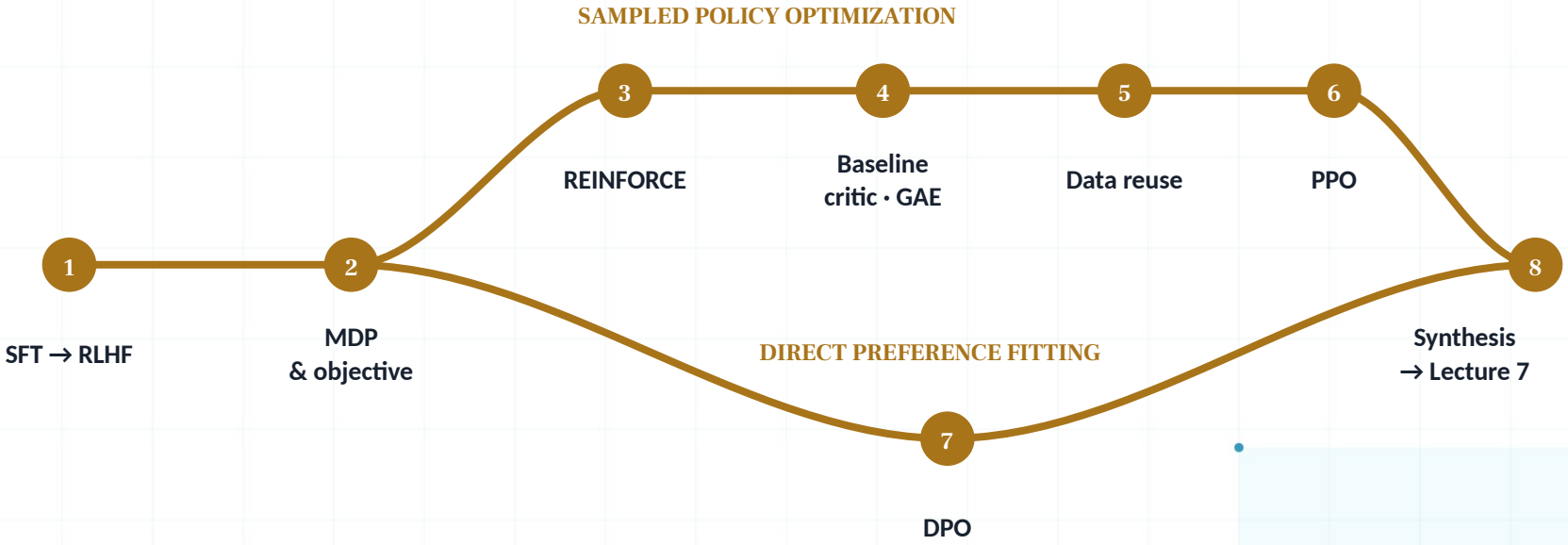
LECTURE 6

Reinforcement Learning for Language Models

From rewards to policy gradients, PPO, and DPO

12 October 2026

Map of the Lecture



Two routes start from the same KL-regularized objective.

01

From SFT to Reinforcement Learning

What changes when the model learns from its own scored answers



Pretraining, SFT, and RL

Pretraining → SFT → RL

Pretraining	SFT	RL
TRAINING SIGNAL	TRAINING SIGNAL	TRAINING SIGNAL
Text corpora	Prompt–answer demonstrations	Prompts + generated answers
Predict the next token	**Fit demonstrated answers**	**Increase expected reward**
A pretrained base model	An initial instruction policy	A policy improved by feedback

A common pipeline; the source of supervision changes at each stage.



Ouyang et al., NeurIPS 2022 · Training language models to follow instructions with human feedback
Zhang et al., 2025 · A Survey of Reinforcement Learning for Large Reasoning Models

Online RL Learns from Generated Responses

SFT: an answer from the dataset

Fit a supplied demonstration y.

$$\mathcal{L}_{\text{SFT}} = -\mathbb{E}_{(x,y)\sim\mathcal{D}} \log \pi_\theta(y \mid x)$$

x: “Write an even number.”
y: “24” supplied by the dataset

RL: an answer sampled by the policy

Improve the expected score of generated answers.

$$J(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta}[R(x, y)]$$



Policy = a distribution over actions. Reward = the feedback used to improve it.

Human Preferences after RLHF

Held-out API prompts; every model is compared with SFT 175B.

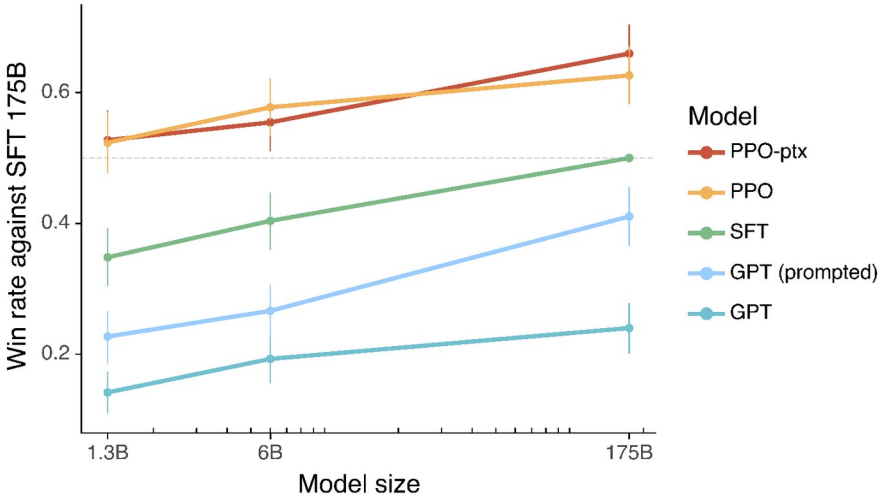


Figure 1 · 95% confidence intervals. RL adds preference feedback beyond SFT.

50% is the reference

A tie with SFT 175B.

PPO-ptx

PPO + a pretraining loss.

Same-size comparison

PPO curves exceed SFT here.

Where Reinforcement Learning Begins

1 Supervised fine-tuning

Human demonstrations

→ **Initial language policy** π_{init}

2 Reward-model training

Human-ranked response pairs

→ **A reusable scalar scorer** $R_\phi(x, y)$

3 Reinforcement learning

New prompts + sampled answers

→ **A policy with higher expected reward**

$$y \sim \pi_\theta(\cdot \mid x) \longrightarrow R_\phi(x, y) \longrightarrow \Delta\theta$$

In this online RLHF loop, the policy generates responses and learns from their scores.



Ouyang et al., NeurIPS 2022 · Training language models to follow instructions with human feedback

Learn a Reward Model from Comparisons

x: “Explain overfitting in one sentence.”

y_w: a concise, correct explanation

y_l: an unrelated answer about model size

PREFERENCE MODEL · BRADLEY–TERRY

$$P_\phi(y_w \succ y_l \mid x) = \sigma(R_\phi(x, y_w) - R_\phi(x, y_l))$$

Equal scores → probability 0.5. A score gap of log 3 → probability 0.75.

$$\mathcal{L}_{\mathrm{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\mathrm{pref}}} \log P_\phi(y_w \succ y_l \mid x)$$

Train the reward model on labeled comparisons; optimize the policy in a separate phase.



Ouyang et al., NeurIPS 2022 · Training language models to follow instructions with human feedback

Rafailov et al., NeurIPS 2023 · Direct Preference Optimization: Your Language Model is Secretly a Reward Model

Score New Responses with a Frozen Reward Model

PROMPT → SAMPLE → SCORE → POLICY UPDATE

$$x \sim \mathcal{D}, \qquad y \sim \pi_\theta(\cdot \mid x), \qquad r = R_\phi(x, y)$$

The policy chooses the answer. The frozen reward model evaluates that answer.

TRAIN THE POLICY PARAMETERS

$$\max_\theta J(\theta), \qquad J(\theta) = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[R_\phi(x, y)]$$

Keep φ fixed. Change θ to raise the average score of newly generated answers.

How can one response score tell us how to update the token probabilities?



Ouyang et al., NeurIPS 2022 · Training language models to follow instructions with human feedback

02

A Policy Acts in an Environment

States, actions, returns, and policy drift



A Finite Episodic Markov Decision Process

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, d_0, T), \qquad \pi_\theta : \mathcal{S} \to \Delta(\mathcal{A})$$

States S · actions A · initial-state distribution d_0 · finite horizon T.

$$s_1 \sim d_0, \quad a_t \sim \pi_\theta(\cdot \mid s_t), \quad s_{t+1} \sim P(\cdot \mid s_t, a_t)$$

$$r_t = r(s_t, a_t, s_{t+1}), \qquad t = 1, \ldots, T$$

P gives a distribution over the next state; r assigns a scalar to the transition.
Terminal states end the episode. Include time in the state when needed.

Markov: given the current state and action, the transition law does not need the earlier history.



Language Generation as an MDP

$$s_t = (x, y_{<t}) \in \mathcal{S}, \qquad a_t = y_t \in \mathcal{A} = \mathcal{V}$$

Policy model: the LLM. State: prompt + response prefix. Action: the next token.
Transition: append that token. EOS or the length limit ends the response.

$$\pi_\theta(a_t \mid s_t) = \pi_\theta(y_t \mid x, y_{<t}), \qquad r_t = \begin{cases} 0, & t < T, \\ R_\phi(x, y), & t = T. \end{cases}$$

MAXIMIZE THE EXPECTED SCORE OF COMPLETE RESPONSES

$$\max_\theta J(\theta), \qquad J(\theta) = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[R_\phi(x, y)]$$

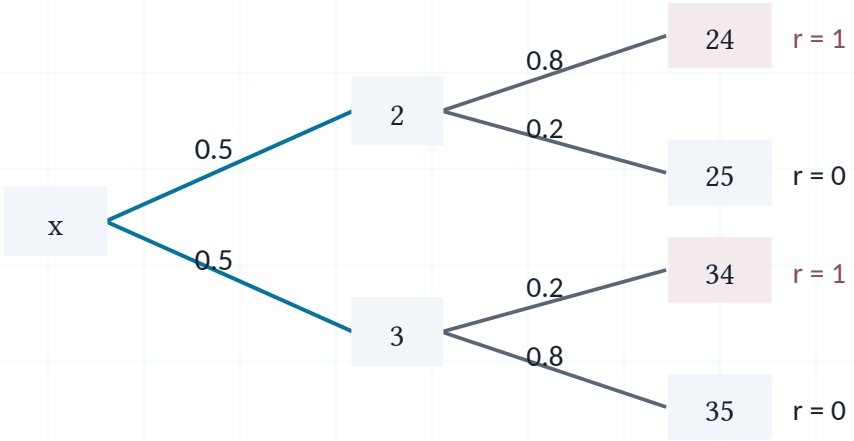
Estimate the expectation by the mean score of sampled prompt–response pairs.



Zhang et al., 2025 · A Survey of Reinforcement Learning for Large Reasoning Models
Ouyang et al., NeurIPS 2022 · Training language models to follow instructions with human feedback

One Prompt, Four Possible Responses

x: “Write a two-digit even number.”



y	π(y | x)	reward
24	0.4	1
25	0.1	0
34	0.1	1
35	0.4	0

Illustrative token policy; deterministic termination after two tokens.

Deterministic Transitions, Stochastic Responses

$$p_\theta(\tau) = d_0(s_1) \prod_{t=1}^{T} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)$$

LLM TRANSITION: THE TOKEN HAS ALREADY BEEN CHOSEN

$$P(s_{t+1} \mid s_t, a_t) = \mathbf{1}\{s_{t+1} = \text{append}(s_t, a_t)\}$$

Append is deterministic. Token selection by the policy is still stochastic.

$$\pi_\theta(y \mid x) = \prod_{t=1}^{T} \pi_\theta(y_t \mid x, y_{<t}), \qquad \pi(34 \mid x) = 0.5 \times 0.2 = 0.1$$

On a valid response path, each transition factor is 1; only the policy factors depend on θ.

Rewards, Returns, and the Objective

$$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k, \qquad G_1 = \text{discounted return of the response}$$

DISCOUNT $\gamma \in [0, 1]$ WEIGHTS LATER REWARDS · HERE $\gamma = 1$

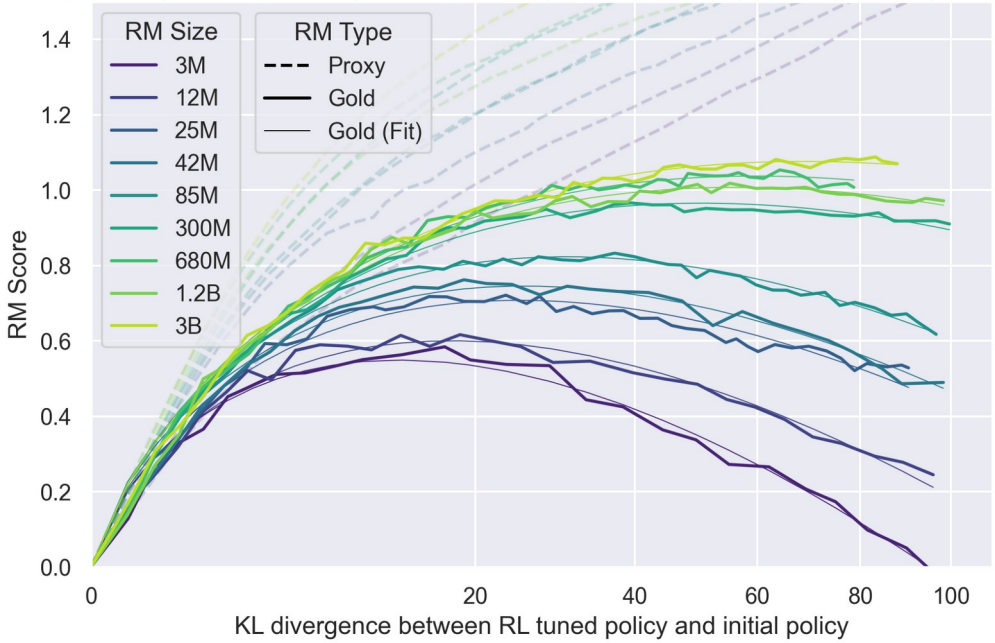
$$x \xrightarrow[r_1=0]{a_1=3} (x, 3) \xrightarrow[r_2=1]{a_2=4} \text{terminal}$$

$$\gamma = 1: \quad G_1 = 0 + 1 = 1, \quad G_2 = 1; \qquad J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[G_1]$$

$$\gamma = 1,\ r_{t<T} = 0: \qquad G(x, y) = R_\phi(x, y), \qquad J = 0.5$$

Treating the whole response y as one action gives the same expected terminal-score objective.

Why Limit Drift from the Reference?



Gold is a synthetic judge. KL controls drift; it does not make the reward model correct.

Gao et al., ICML 2023 · Scaling Laws for Reward Model Overoptimization

Follow the purple 3M pair

Dashed: optimized proxy.
Solid: gold evaluation score.
x: KL from the initial model.

A reason to regularize

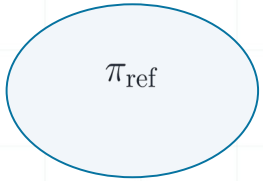
Higher proxy score can coincide with lower gold score. Charge for drift from the reference.

The KL-Regularized Policy Objective

$$K(\theta) = \mathbb{E}_{x\sim\mathcal{D}}[D_{\mathrm{KL}}(\pi_\theta(\cdot \mid x)\|\pi_{\mathrm{ref}}(\cdot \mid x))]$$

$$K(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\left[\log \frac{\pi_\theta(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)}\right]$$

Average prompts from D and responses from the current policy. The reference is fixed.



Schematic set K(θ) ≤ c

REGULARIZED OBJECTIVE · β ▪ 0

$$\max_\theta \; J_\beta(\theta) = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[R_\phi(x,y)] - \beta K(\theta)$$

β is the price of drifting from the fixed reference.

Zhang et al., 2025 · A Survey of Reinforcement Learning for Large Reasoning Models
Rafailov et al., NeurIPS 2023 · Direct Preference Optimization: Your Language Model is Secretly a Reward Model

Sequence KL from Token Log-Probabilities

Why this decomposition? We already compute token log-probabilities during an LLM forward pass.

$$\log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} = \sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})}$$

1. Sample a response from the current policy.
2. Score the same tokens under the actor and the frozen reference.
3. Sum the token log-ratios; average across sampled prompt–response pairs.

One sampled log-ratio may be negative. Its current-policy expectation is a nonnegative KL.



Ouyang et al., NeurIPS 2022 · Training language models to follow instructions with human feedback
Rafailov et al., NeurIPS 2023 · Direct Preference Optimization: Your Language Model is Secretly a Reward Model

03

Differentiate an Expected Return

From a sampled trajectory to a policy gradient



The Sampling Distribution Depends on θ

$$J_x(\theta) = \mathbb{E}_{y \sim p_\theta(\cdot|x)}[G(x,y)] = \Sigma_y \, p_\theta(y \mid x) G(x,y)$$

$$p_\theta(y \mid x) = \Pi_{t=1}^{T} \pi_\theta(y_t \mid x, y_{<t}), \qquad J(\theta) = \mathbb{E}_{x \sim \mathcal{D}}[J_x(\theta)]$$

DIFFERENTIATION THROUGH A CHANGING DISTRIBUTION

A sampled token is discrete. A fixed reward need not have a derivative with respect to the text.

$$\nabla_\theta J_x = \Sigma_y \, G(x,y) \nabla_\theta p_\theta(y \mid x)$$

Fix a prompt and response. Its score is fixed; the probability of generating it depends on θ.



Zhang et al., 2025 · A Survey of Reinforcement Learning for Large Reasoning Models
Sutton et al., NeurIPS 1999 · Policy Gradient Methods for Reinforcement Learning with Function Approximation

The Log-Derivative Trick

$$\nabla_\theta J_x = \Sigma_y G(x,y) \nabla_\theta p_\theta(y \mid x)$$

1 Differentiate response probabilities

$$= \Sigma_y p_\theta(y \mid x) G(x,y) \frac{\nabla_\theta p_\theta(y|x)}{p_\theta(y|x)}$$

2 Multiply and divide by the response probability

$$= \Sigma_y p_\theta(y \mid x) G(x,y) \nabla_\theta \log p_\theta(y \mid x)$$

3 Use the derivative of log probability

$$= \mathbb{E}_{y \sim p_\theta(\cdot|x)} [G(x,y) \nabla_\theta \log p_\theta(y \mid x)]$$

4 Sample responses to estimate this expectation

Exact identity for a fixed reward. Sampling approximates the expectation, not the calculus.



Williams, Machine Learning 1992 · Simple Statistical Gradient-Following Algorithms for Connectionist RL

Sutton et al., NeurIPS 1999 · Policy Gradient Methods for Reinforcement Learning with Function Approximation

Only Policy Terms Need a Gradient

EXPAND THE LOG-PROBABILITY OF A TRAJECTORY

$$\log p_\theta(\tau) = \log d_0(s_1) + \Sigma_t[\log \pi_\theta(a_t \mid s_t) + \log P(s_{t+1} \mid s_t, a_t)]$$

d_0 and P do not depend on θ. Only the policy factors contribute to the gradient.

DIFFERENTIATE

$$\nabla_\theta \log p_\theta(\tau) = \Sigma_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

For an LLM, compute the log-probability of each sampled token given its saved prefix.



Policy Gradient Uses Future Rewards

$$u_t = \nabla_\theta \log \pi_\theta(a_t \mid s_t), \qquad \nabla_\theta \log p_\theta(\tau) = \Sigma_t\, u_t, \qquad \gamma = 1$$

$$\nabla J = \mathbb{E}[G_1 \,\Sigma_t\, u_t] = \mathbb{E}[\Sigma_t\, u_t\, (\Sigma_{k<t}\, r_k + \Sigma_{k\geq t}\, r_k)]$$

h_t is the history of states, past actions and observed rewards before action a_t.

For an LLM, the state contains the prompt and the tokens generated so far.

$$\mathbb{E}[u_t \mid h_t] = \Sigma_a\, \nabla_\theta \pi_\theta(a \mid s_t) = 0 \quad \Longrightarrow \quad \mathbb{E}[u_t\, \Sigma_{k<t}\, r_k] = 0$$

$$\nabla_\theta J = \mathbb{E}_{\tau\sim p_\theta}\left[\Sigma_t \quad \underbrace{G_t}_{\text{future rewards}} \quad \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$$

Williams, Machine Learning 1992 · Simple Statistical Gradient-Following Algorithms for Connectionist RL
Sutton et al., NeurIPS 1999 · Policy Gradient Methods for Reinforcement Learning with Function Approximation

REINFORCE: One Complete Update

LLM setting: terminal-only reward, γ = 1. Train only on response tokens.

1 Generate

$$x^{(i)} \sim \mathcal{D}, \quad y^{(i)} \sim p_\theta(\cdot \mid x^{(i)}), \quad i = 1, \ldots, N$$

2 Score

$$R_i = R(x^{(i)}, y^{(i)}), \quad G_t^{(i)} = R_i \quad (t = 1, \ldots, T_i)$$

3 Update

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\mathrm{PG}}, \qquad G_t^{(i)} \text{ fixed}$$

AVERAGE OVER RESPONSES; SUM OVER THEIR TOKENS

$$\mathcal{L}_{\mathrm{PG}} = -\frac{1}{N} \Sigma_{i=1}^{N} \Sigma_t \, \text{stop-grad}[G_t^{(i)}] \log \pi_\theta(y_t^{(i)} \mid x^{(i)}, y_{<t}^{(i)})$$

4 Refresh

$$x^{(i)} \sim \mathcal{D}, \qquad y^{(i)} \sim p_{\theta_{\text{updated}}}(\cdot \mid x^{(i)})$$



Williams, Machine Learning 1992 · Simple Statistical Gradient-Following Algorithms for Connectionist RL

A Successful Answer Can Reinforce a Poor Choice

Toy task: generate a two-digit even number. Both first tokens are sampled with probability 0.5.

First token “2”

Next token “4” with probability 0.8
Expected success: 80%

First token “3”

Next token “4” with probability 0.2
Expected success: 20%

Suppose we sample the lucky answer “34”. Its terminal score is 1, so both token returns are 1.

$$\hat{g}_{34} = \nabla_\theta \log \pi_\theta(3 \mid x) + \nabla_\theta \log \pi_\theta(4 \mid x, 3)$$

This sample reinforces “3” even though “2” succeeds more often. We need an expectation and a baseline.



Williams, Machine Learning 1992 · Simple Statistical Gradient-Following Algorithms for Connectionist RL

04

Assign Credit with a Baseline

Expected futures, bootstrapping, and advantage estimates



Values Compare Expected Futures

STATE VALUE

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

Choose the next action, then follow π.

ACTION VALUE

$$Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$$

Fix this action, then follow π.

ADVANTAGE

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

Compare this action with the policy average.

At the first token

$$V^\pi(x) = 0.5$$

$$Q^\pi(x,2) = 0.8, \quad A^\pi = +0.3$$

$$Q^\pi(x,3) = 0.2, \quad A^\pi = -0.3$$

Advantage measures how much better an action is than the current policy at that state.



Zhang et al., 2025 · A Survey of Reinforcement Learning for Large Reasoning Models

Sutton et al., NeurIPS 1999 · Policy Gradient Methods for Reinforcement Learning with Function Approximation

Why Subtract a Baseline?

A sampled return is noisy. Center its weight while preserving the expected policy gradient.

$$\hat{g}_t = G_t u_t \quad \longrightarrow \quad \hat{g}_t^b = (G_t - b(s_t))u_t$$

Response	Probability	No baseline	Baseline 0.5
24 / 25	0.4 / 0.1	+0.50 / 0	+0.25 / −0.25
34 / 35	0.1 / 0.4	−0.50 / 0	−0.25 / +0.25
Mean / variance		0.15 / 0.1025	0.15 / 0.0400

Toy root: π(2) = sigmoid(z) = 0.5. Table entries are gradient samples with respect to z.



Williams, Machine Learning 1992 · Simple Statistical Gradient-Following Algorithms for Connectionist RL

A Baseline Preserves the Mean Gradient

Fix a prefix s. The same baseline b(s) applies to every possible next token.

$$\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[b(s)\nabla_\theta \log \pi_\theta(a \mid s)] = b(s)\sum_a \pi_\theta(a \mid s)\frac{\nabla_\theta \pi_\theta(a \mid s)}{\pi_\theta(a \mid s)}$$

$$= b(s)\nabla_\theta \underbrace{\sum_a \pi_\theta(a \mid s)}_{1} = 0.$$

CENTERED SAMPLES, SAME EXPECTED GRADIENT

$$\mathbb{E}[(G_t - b(s_t))u_t] = \mathbb{E}[G_t u_t] - 0$$

In the actor loss, stop the gradient through the baseline. Its value may be learned separately.



Williams, Machine Learning 1992 · Simple Statistical Gradient-Following Algorithms for Connectionist RL

Sutton et al., NeurIPS 1999 · Policy Gradient Methods for Reinforcement Learning with Function Approximation

From a Baseline to the Advantage

$$b(s_t) = V^\pi(s_t) = \mathbb{E}_\pi[G_t \mid s_t]$$

$$\mathbb{E}_\pi[G_t - V^\pi(s_t) \mid s_t, a_t] = \underbrace{Q^\pi(s_t, a_t) - V^\pi(s_t)}_{A^\pi(s_t, a_t)}$$

EXPECTED POLICY GRADIENT WITH ADVANTAGE

$$\nabla_\theta J = \mathbb{E}_{\tau \sim p_\theta}[\Sigma_t \, A^\pi(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t \mid s_t)]$$

Toy root: token “2” gives 0.8 − 0.5 = +0.3. Token “3” gives 0.2 − 0.5 = −0.3.



Williams, Machine Learning 1992 · Simple Statistical Gradient-Following Algorithms for Connectionist RL
Sutton et al., NeurIPS 1999 · Policy Gradient Methods for Reinforcement Learning with Function Approximation

The Critic Is a Learned Value Model

Model	Input	Output
Actor	$(x, y_{<t})$ Prompt + response prefix	$\pi_\theta(\cdot \mid x, y_{<t}) \in \Delta(\mathcal{V})$ Next-token distribution
Value critic	$(x, y_{<t})$ Prompt + response prefix	$V_\psi(x, y_{<t}) \in \mathbb{R}$ Expected future return
Reward model	(x, y) Prompt + full response	$R_\phi(x, y) \in \mathbb{R}$ Score of a complete response

$$V_\psi(x, y_{<t}) \approx V^\pi(s_t) = \mathbb{E}_\pi[G_t \mid s_t = (x, y_{<t})]$$



Zhang et al., 2025 · A Survey of Reinforcement Learning for Large Reasoning Models
Schulman et al., ICLR 2016 · High-Dimensional Continuous Control Using Generalized Advantage Estimation

Observed Returns Train the Value Model

RANDOM RETURN VS. OBSERVED VALUE ON RESPONSE i

$$G_t = \Sigma_{k=t}^{T} \gamma^{k-t} r_k, \qquad G_t^{(i)} = \Sigma_{k=t}^{T_i} \gamma^{k-t} r_k^{(i)}$$

Generate a response, score it, and regress each prefix value to its observed return.

MONTE CARLO TARGET FOR THE UNKNOWN EXPECTATION Vπ

$$\mathcal{L}_V(\psi) = \frac{1}{N} \Sigma_{i,t} \left(V_\psi(s_t^{(i)}) - \text{stop-grad}[G_t^{(i)}] \right)^2$$

Prefix “3”: targets 1 and 0 occur with probabilities 0.2 and 0.8. Their mean is Vπ = 0.2.



Zhang et al., 2025 · A Survey of Reinforcement Learning for Large Reasoning Models
Schulman et al., ICLR 2016 · High-Dimensional Continuous Control Using Generalized Advantage Estimation

Bellman Identity and the TD Target

Split the future return into the next reward and the remaining future.

$$G_t = r_t + \gamma G_{t+1}$$

BELLMAN EXPECTATION · EXACT VALUE FUNCTION

$$V^\pi(s_t) = \mathbb{E}_\pi[r_t + \gamma V^\pi(s_{t+1}) \mid s_t]$$

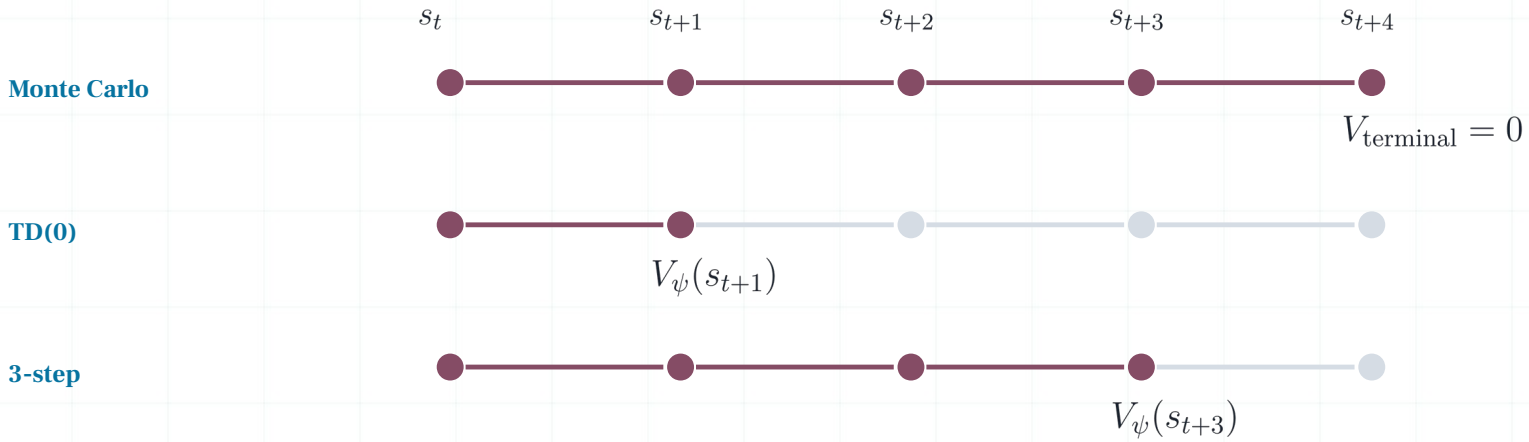
TD(0) · ONE OBSERVED TRANSITION ✦ A CRITIC PREDICTION

$$\widehat{G}_t^{(1)} = r_t + \gamma\,\text{stop-grad}[V_\psi(s_{t+1})]$$

The sampled TD target bootstraps from the critic. At a true terminal state, the next value is zero.



Monte Carlo, TD, and n-Step Returns



$$\widehat{G}_t^{(n)} = \sum_{\ell=0}^{n-1} \gamma^\ell r_{t+\ell} + \gamma^n V_\psi(s_{t+n})$$

Observe n rewards, then bootstrap from the value after n transitions.

Schulman et al., ICLR 2016 · High-Dimensional Continuous Control Using Generalized Advantage Estimation

One-Step TD Estimates Advantage

Advantage compares this token with the policy average at the same prefix.

$$A^{\pi}(s_t, a_t) = \underbrace{Q^{\pi}(s_t, a_t)}_{\text{after choosing this token}} - \underbrace{V^{\pi}(s_t)}_{\text{before choosing a token}}$$

ONE OBSERVED TRANSITION ✦ A VALUE PREDICTION

$$\widehat{A}_t^{(1)} = \delta_t = \underbrace{r_t + \gamma V_{\psi}(s_{t+1})}_{\text{one-step estimate of } Q^{\pi}(s_t, a_t)} - V_{\psi}(s_t)$$

$$V_{\psi} = V^{\pi} \implies \mathbb{E}[\delta_t \mid s_t, a_t] = A^{\pi}(s_t, a_t)$$

Two TD Errors Give a Two-Step Estimate

$$V_t := V_\psi(s_t), \qquad \delta_t := r_t + \gamma V_{t+1} - V_t$$

$$\delta_t + \gamma\delta_{t+1} = (r_t + \gamma V_{t+1} - V_t) + (\gamma r_{t+1} + \gamma^2 V_{t+2} - \gamma V_{t+1})$$

The highlighted terms cancel: one more observed step replaces a value prediction.

A TWO-STEP ESTIMATE OF THE SAME STARTING ACTION

$$\widehat{A}_t^{(2)} = \underbrace{r_t + \gamma r_{t+1} + \gamma^2 V_{t+2}}_{\widehat{G}_t^{(2)}\ :\ \text{two-step return target}} - V_t$$

More future evidence, the same starting token and baseline.

From Two Horizons to Three

$$\widehat{A}_t^{(1)} = \delta_t, \qquad \widehat{A}_t^{(2)} = \delta_t + \gamma\delta_{t+1}$$

$$\begin{aligned}\delta_t + \lambda\gamma\delta_{t+1} &= (1-\lambda)\delta_t + \lambda(\delta_t + \gamma\delta_{t+1}) \\ &= (1-\lambda)\widehat{A}_t^{(1)} + \lambda\widehat{A}_t^{(2)}\end{aligned}$$

One more step: $\widehat{A}_t^{(3)} = \widehat{A}_t^{(2)} + \gamma^2\delta_{t+2}$

THREE HORIZONS: THE WEIGHTS SUM TO ONE

$$\begin{aligned}&\delta_t + \gamma\lambda\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} \\ &\quad = (1-\lambda)\widehat{A}_t^{(1)} + \lambda(1-\lambda)\widehat{A}_t^{(2)} + \lambda^2\widehat{A}_t^{(3)}\end{aligned}$$

The last horizon keeps the remaining weight: λ^2, not $(1-\lambda)\lambda^2$.

Multi-Step Advantage and GAE

$$\widehat{A}_t^{(1)} = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t) = \delta_t$$

$$\widehat{A}_t^{(2)} = r_t + \gamma r_{t+1} + \gamma^2 V_\psi(s_{t+2}) - V_\psi(s_t) = \delta_t + \gamma\delta_{t+1}$$

More steps replace value predictions with observed rewards before the final bootstrap.

MIX SHORT AND LONG HORIZONS

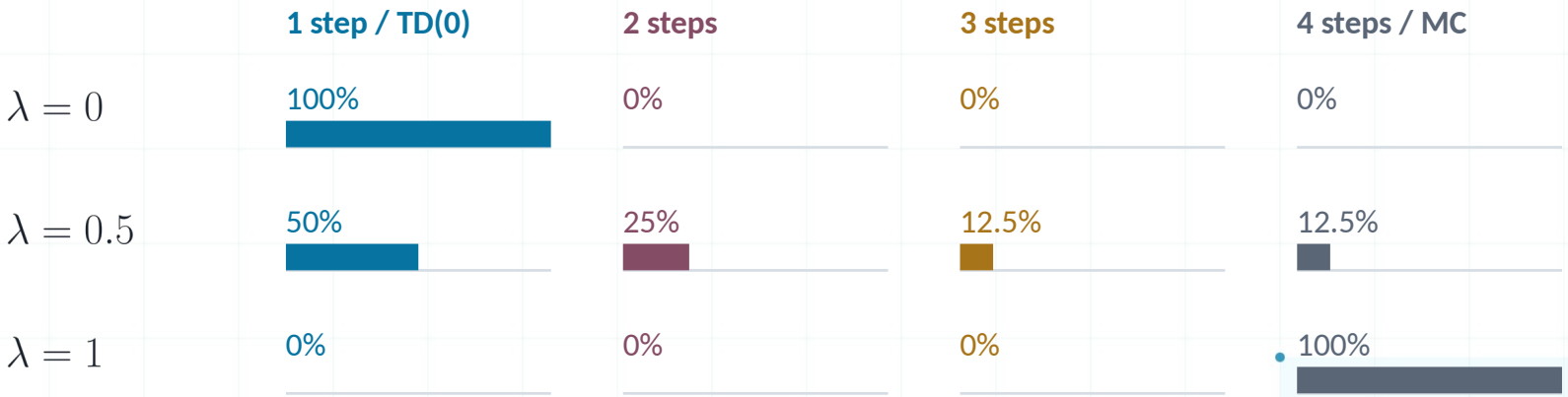
$$\widehat{A}_t^{\mathrm{GAE}} = \Sigma_{\ell=0}^{T-t}(\gamma\lambda)^\ell \delta_{t+\ell}$$

$$\widehat{A}_t = \delta_t + \gamma\lambda\widehat{A}_{t+1}, \qquad \widehat{A}_{T+1} = 0$$



How λ Mixes the TD Horizons

Four transitions remain. Bar lengths show weights of the n-step advantage estimates.



$$\lambda = 0 : \ \widehat{A}_t = \delta_t, \qquad \lambda = 1 : \ \widehat{A}_t = G_t^{(i)} - V_\psi(s_t)$$

GAE from Learned Prefix Values

Prompt x

V-model = 0.4 | next reward = 0

Prefix “3”

V-model = 0.3 | next reward = 1

Terminal

V-model = 0 | next reward = —

$$\gamma = 1: \quad \delta_2 = 1 + 0 - 0.3 = 0.7, \qquad \delta_1 = 0 + 0.3 - 0.4 = -0.1$$

APPROXIMATE CRITIC: $V_\psi(x) = 0.4$, $V_\psi(x, 3) = 0.3$

$$\widehat{A}_2 = 0.7, \qquad \widehat{A}_1 = -0.1 + 0.7\lambda$$

λ = 0 → −0.10 **λ = 0.5 → +0.25** **λ = 1 → +0.60**

The Choice of λ Changes Training

PPO · Llama-7B-based models · Anthropic HH

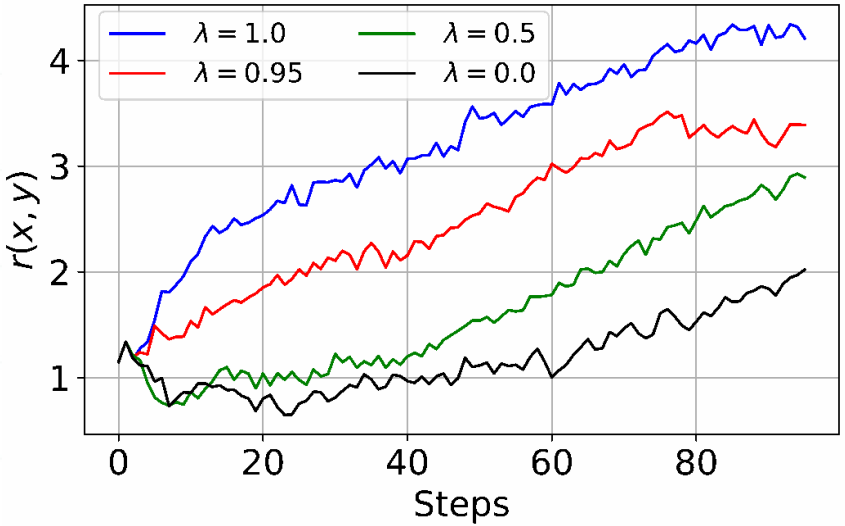


Figure 1, left · Reducing estimator variance can still hurt when it introduces bias.

What is measured?

Training reward from the reward model.

What happens here?

λ = 1 reaches the highest reward.

What does not follow?

There is no universal best λ.

An On-Policy Actor–Critic Update

1 Sample Generate answers with the current actor; save tokens and log-probabilities.

2 Score The frozen reward model gives terminal scores; the critic predicts prefix values.

3 Estimate Compute TD residuals and GAE backwards; freeze advantages and value targets.

4 UPDATE THE ACTOR; FIT THE CRITIC TO FROZEN RETURN TARGETS

$$\mathcal{L}_{\text{actor}} = -\frac{1}{N}\sum_{i,t} \text{stop-grad}[\widehat{A}_t^{(i)}] \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)})$$

$$\widehat{G}_t = \widehat{A}_t + V_{\text{old}}(s_t)$$

Use critic values saved before the update.
Freeze targets, fit the critic, collect fresh responses.

Schulman et al., ICLR 2016 · High-Dimensional Continuous Control Using Generalized Advantage Estimation
Sutton et al., NeurIPS 1999 · Policy Gradient Methods for Reinforcement Learning with Function Approximation

05

Which Policy Generated the Data?

Data collection, distribution mismatch, and importance weights



Two Separate Questions about the Training Data

1 · CAN WE COLLECT NEW RESPONSES?

offline: fixed dataset online: new interactions

Online describes the training loop's access to new data, not whether the estimator is on-policy.

2 · WHICH POLICY GENERATED THIS BATCH?

on-policy: $\pi_b = \pi_\theta$ off-policy: $\pi_b \neq \pi_\theta$

Compare the behavior policy that generated the batch with the policy whose return we optimize.

An online loop can reuse old responses and perform off-policy updates.



Tajwar et al., ICML 2024 · Preference Fine-Tuning of LLMs Should Leverage Suboptimal, On-Policy Data

Behavior Policy Collects; Current Policy Learns

BEHAVIOR POLICY

$\pi_b = \pi_{\theta_{\text{old}}}$

Generates this batch once.
Its sampled-action log-probs stay fixed.

CURRENT ACTOR

π_θ

Changes after every optimizer step.
Recompute its log-probs on saved tokens.

The buffer contains fixed responses. The policy being optimized is a moving distribution.

MEASURE THE MISMATCH ON EACH SAVED ACTION

$$w_t(\theta) = \frac{\pi_\theta(y_t|x,y_{<t})}{\pi_b(y_t|x,y_{<t})}$$



Zhang et al., 2025 · A Survey of Reinforcement Learning for Large Reasoning Models
Schulman et al., 2017 · Proximal Policy Optimization Algorithms

Importance Weights Reweight the Advantage

The batch comes from the behavior policy. Reweight its saved advantages for the current actor.

$$s = (x, y_{<t}), \qquad w(a \mid s) = \frac{\pi_\theta(a|s)}{\pi_b(a|s)}$$

EXACT CHANGE OF NEXT-TOKEN DISTRIBUTION AT ONE FIXED PREFIX

$$\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[A^{\pi_b}(s,a)] = \mathbb{E}_{a\sim\pi_b(\cdot|s)}[w(a \mid s)A^{\pi_b}(s,a)]$$

$$0.7(0.3) + 0.3(-0.3) = 0.5(1.4)(0.3) + 0.5(0.6)(-0.3) = 0.12$$

The weights correct sampling frequencies. The advantage still refers to the behavior policy.



Zhang et al., 2025 · A Survey of Reinforcement Learning for Large Reasoning Models

Schulman et al., 2017 · Proximal Policy Optimization Algorithms

A Whole Response Needs a Product of Weights

$$\mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[F(x,y)] = \mathbb{E}_{y\sim\pi_b(\cdot|x)}[W(x,y)F(x,y)]$$

$$W(x,y) = \frac{\pi_\theta(y|x)}{\pi_b(y|x)} = \Pi_{t=1}^{T} \frac{\pi_\theta(y_t|x,y_{<t})}{\pi_b(y_t|x,y_{<t})}$$

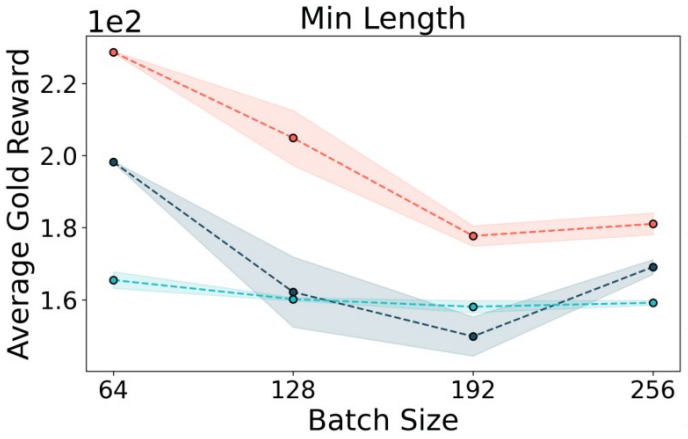
The behavior policy must cover the current policy's support. Long products can have extreme variance.

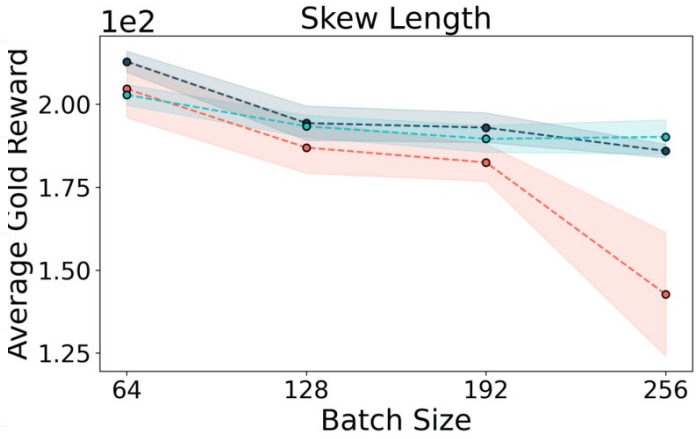
Even $1.01^{1000} \approx 20{,}959$. PPO uses a local token-weighted surrogate instead.

A token ratio corrects the next-token distribution; it does not correct which prefixes were visited.

More Frequent Rollouts Can Improve Reward

x: rollout collection batch size B (not SGD minibatch size).
y: mean gold reward during training; read the ×100 multiplier.

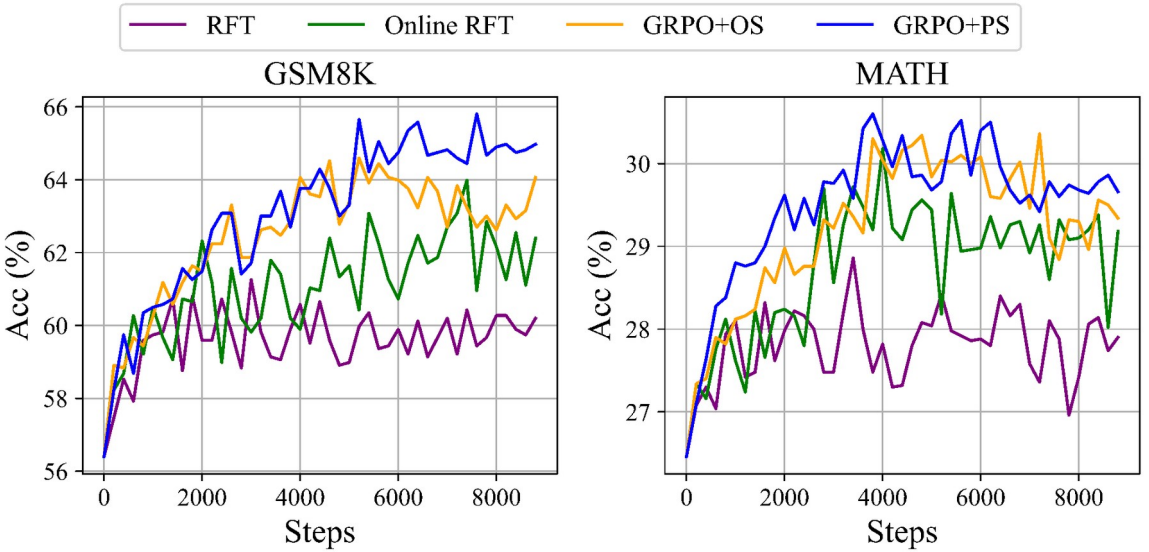




PPO · On-policy RWR · REINFORCE

Learning from Fresh Responses

Same initial model: DeepSeekMath-Instruct 1.3B.



RFT

Fixed SFT samples; keep correct answers.

Online RFT

Fresh policy samples; keep correct answers.

GRPO + OS

Reward for the whole answer.

GRPO + PS

Rewards for reasoning steps.

Figure 5 · RFT uses correctness rules; GRPO uses a learned reward model.

Shao et al., 2024 · DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

06

Proximal Policy Optimization

A local surrogate and controlled reuse of sampled responses



Three Policies, Two Different Comparisons

CURRENT ACTOR

π_θ

Changes at every optimizer step

ROLLOUT POLICY

$\pi_{\theta_{\text{old}}}$

Fixed while this batch is reused

REFERENCE

π_{ref}

Fixed throughout the RL phase

$$w_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

Ratio: change since data collection.

$$D_{\text{KL}}(\pi_\theta(\cdot \mid s_t) \| \pi_{\text{ref}}(\cdot \mid s_t))$$

KL: drift from the fixed anchor.

Refresh the rollout policy for the next batch. Keep the reference unchanged.



Schulman et al., 2017 · Proximal Policy Optimization Algorithms
Ouyang et al., NeurIPS 2022 · Training language models to follow instructions with human feedback

A Local Surrogate for Policy Improvement

Freeze old prefixes and advantages. sg means stop-gradient; differentiate only the current ratio.

OLD PREFIXES ✦ OLD ADVANTAGES ✦ CURRENT / OLD TOKEN WEIGHTS

$$\mathcal{J}_{\mathrm{surr}}(\theta) = \mathbb{E}_{\tau \sim p_{\theta_{\mathrm{old}}}}\left[\sum_t w_t(\theta)\, \mathrm{sg}\left[A^{\pi_{\theta_{\mathrm{old}}}}(s_t, a_t)\right]\right]$$

$$w_t|_{\theta_{\mathrm{old}}} = 1, \qquad \nabla_\theta w_t|_{\theta_{\mathrm{old}}} = \nabla_\theta \log \pi_\theta(a_t \mid s_t)|_{\theta_{\mathrm{old}}}$$

EXACT LOCAL GRADIENT MATCH WITH TRUE ADVANTAGES

$$\nabla \mathcal{J}_{\mathrm{surr}}(\theta_{\mathrm{old}}) = \nabla J(\theta_{\mathrm{old}})$$

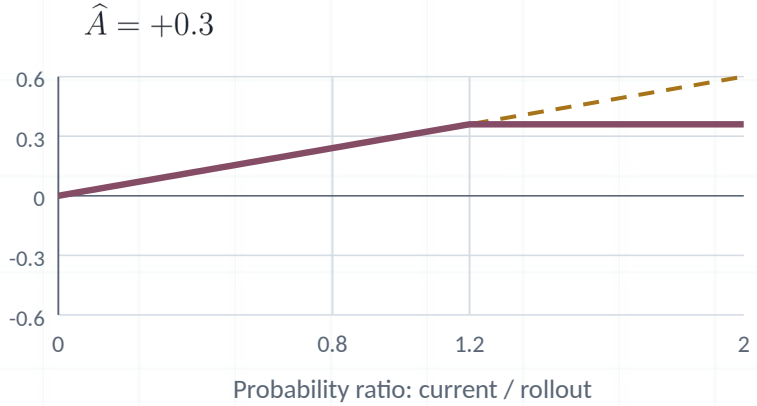


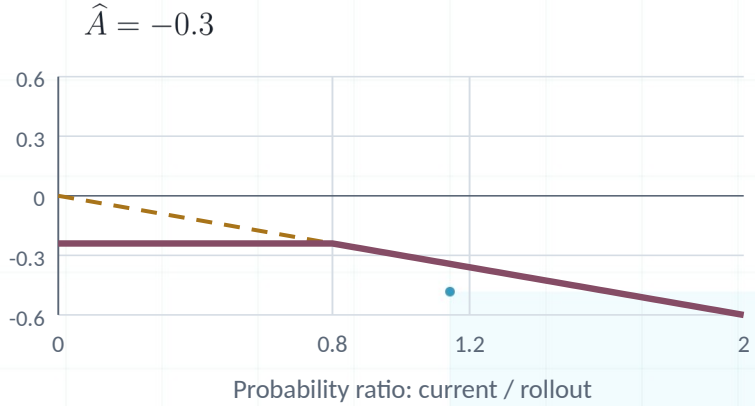
Zhang et al., 2025 · A Survey of Reinforcement Learning for Large Reasoning Models
Schulman et al., 2017 · Proximal Policy Optimization Algorithms

PPO Clips Large Favorable Changes

$$\mathcal{J}_{\mathrm{PPO}}(\theta) = \mathbb{E}_{\mathrm{old}} \sum_t \min\left(w_t \widehat{A}_t, \mathrm{clip}(w_t, 1-\varepsilon, 1+\varepsilon)\widehat{A}_t\right)$$

Vertical axis: objective contribution · ε = 0.2 · dashed: unclipped · solid: PPO





Same axis scales: positive advantages plateau above 1.2; negative advantages plateau below 0.8.

Zhang et al., 2025 · A Survey of Reinforcement Learning for Large Reasoning Models
Schulman et al., 2017 · Proximal Policy Optimization Algorithms

When Does Clipping Remove the Incentive?

POSITIVE ADVANTAGE: INCREASE THE ACTION PROBABILITY

$$\widehat{A} = +0.3 : \quad \min(w\widehat{A}, \mathrm{clip}(w)\widehat{A}) = 0.3 \min(w, 1.2)$$

Old probability 0.20 → upper threshold 0.24. Raising it to 0.28 gives no extra benefit.

NEGATIVE ADVANTAGE: DECREASE THE ACTION PROBABILITY

$$\widehat{A} = -0.3 : \quad \min(w\widehat{A}, \mathrm{clip}(w)\widehat{A}) = -0.3 \max(w, 0.8)$$

Old probability 0.20 → lower threshold 0.16. Lowering it to 0.12 gives no extra benefit.

With ε = 0.2, only favorable movement plateaus. Moving in the wrong direction still receives a gradient.



Clipping, Reference KL, and Entropy

Mechanism	What it compares	Purpose
PPO clipping · ε	current / rollout likelihood	Limit incentives for large favorable changes
Reference KL · β	current vs. fixed reference	Penalize drift from an anchor
Entropy bonus	current action distribution	Encourage stochasticity

Clipping gives neither a global KL bound nor a monotonic-improvement guarantee.



Schulman et al., 2017 · Proximal Policy Optimization Algorithms

Gao et al., ICML 2023 · Scaling Laws for Reward Model Overoptimization

The Complete PPO Training Loss

MINIMIZE OVER THE ACTOR θ AND VALUE MODEL ψ

$$\mathcal{L}(\theta,\psi) = \frac{1}{N}\sum_{i,t}\Big[-\min\Big(w_t\,\mathrm{sg}[\widehat{A}_t],\ \mathrm{clip}(w_t,1-\varepsilon,1+\varepsilon)\,\mathrm{sg}[\widehat{A}_t]\Big) + \beta D_{\mathrm{KL}}(\pi_\theta(\cdot \mid s_t)\|\pi_{\mathrm{ref}}(\cdot \mid s_t)) + c_V\left(V_\psi(s_t) - \mathrm{sg}[\widehat{G}_t]\right)^2\Big].$$

Three terms: policy improvement, reference penalty, value regression.

$$w_t = \pi_\theta(a_t \mid s_t)/\pi_{\mathrm{old}}(a_t \mid s_t), \quad \widehat{G}_t = \widehat{A}_t + V_{\mathrm{old}}(s_t), \quad \mathrm{sg} = \text{stop-gradient}$$



Ouyang et al., NeurIPS 2022 · Training language models to follow instructions with human feedback
Schulman et al., 2017 · Proximal Policy Optimization Algorithms

PPO–RLHF: Roles in One Training Iteration

1 Rollout policy	Freeze the actor snapshot; generate responses and save old log-probs.
2 Reward model + critic	Score complete responses; predict prefix values; form external returns and GAE.
3 Current actor + reference	Recompute current log-probs and ratios; evaluate KL to the frozen reference.
4 Actor + critic update	Maximize clipped advantage − β KL; fit the critic to frozen value targets.
5 Refresh rollout policy	Copy the updated actor for the next collection round. Keep the reference fixed.



Ouyang et al., NeurIPS 2022 · Training language models to follow instructions with human feedback
Schulman et al., 2017 · Proximal Policy Optimization Algorithms

Practical Limits of PPO for LLMs

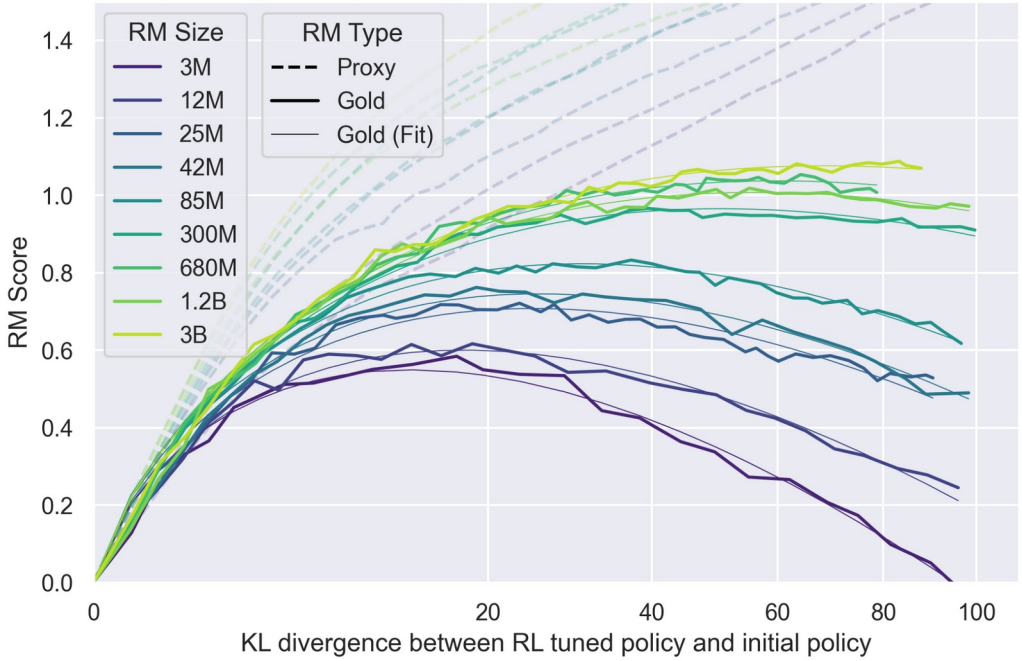
Bottleneck	Consequence	What to monitor
Rollout generation	Long responses cost time	End-to-end cost per useful update
Value prediction	A changing target for the critic	Value error and advantage scale
Batch reuse	The actor drifts from its sampler	Ratios, KL, update epochs
Proxy reward	High score can hide worse answers	Independent task quality

PPO needs fresh data and a reliable learning signal, in addition to clipping.



Ouyang et al., NeurIPS 2022 · Training language models to follow instructions with human feedback
Schulman et al., 2017 · Proximal Policy Optimization Algorithms

Reward Overoptimization in PPO



The PPO failure mode

1.2B policy; synthetic gold judge.
Optimization can exploit proxy errors.

Read one color

Dashed: optimized RM.
Solid: independent gold judge.
Color: proxy RM size.
x: KL from the start (√-scaled).

After the solid curve peaks, more proxy reward can mean worse evaluation.

Gao et al., ICML 2023 · Scaling Laws for Reward Model Overoptimization

07

Direct Preference Optimization

Use the optimal policy to parameterize preference likelihoods



DPO: From Preferences to a Policy

$$\mathcal{D}_{\text{pref}} = \{(x, y_w, y_l)\}, \qquad y_w \succ y_l$$

We observe which response wins. We do not observe a numerical reward for every response.

STARTING POINT: THE SAME KL-REGULARIZED GOAL, FOR ONE PROMPT

$$\max_\pi \ \mathbb{E}_{y\sim\pi(\cdot|x)}[R(x,y)] - \beta D_{\text{KL}}(\pi(\cdot \mid x)\|\pi_{\text{ref}}(\cdot \mid x))$$

1 Solve for π* given R. 2 Express R through π*.
3 Substitute into pair preferences. 4 Fit πθ to the observed labels.

1. Solve the Constrained Policy Problem

$$p_y = \pi(y \mid x), \quad q_y = \pi_{\text{ref}}(y \mid x) > 0, \quad R_y = R(x, y); \qquad p_y \geq 0,\ \Sigma_y\, p_y = 1$$

$$\mathcal{L}(p, \eta) = \Sigma_y\, p_y \Big[R_y - \beta \log \tfrac{p_y}{q_y} \Big] + \eta(\Sigma_y\, p_y - 1)$$

$$0 = \frac{\partial \mathcal{L}}{\partial p_y} = R_y - \beta\Big(\log \tfrac{p_y}{q_y} + 1\Big) + \eta \quad \Longrightarrow \quad p_y = C q_y e^{R_y/\beta}$$

$$1 = \Sigma_y\, p_y = C \underbrace{\sum_y q_y e^{R_y/\beta}}_{Z(x)} \quad \Longrightarrow \quad C = \tfrac{1}{Z(x)}$$

$$\boxed{\pi^*(y \mid x) = \frac{\pi_{\text{ref}}(y \mid x) e^{R(x,y)/\beta}}{Z(x)}}$$

2. Express Reward through the Policy

TAKE LOGS OF THE OPTIMAL-POLICY FORMULA

$$\beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} = R(x,y) - \beta \log Z(x)$$

REWARD IS A POLICY LOG-RATIO, UP TO A PROMPT-ONLY SHIFT

$$R(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \underbrace{\beta \log Z(x)}_{\text{same for every response to } x}$$

REPLACE UNKNOWN π* BY TRAINABLE πθ TO DEFINE AN IMPLICIT REWARD

$$R_\theta(x,y) := \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$$

Parameterize the implicit reward with πθ. The next step connects it to observed preferences.



Rafailov et al., NeurIPS 2023 · Direct Preference Optimization: Your Language Model is Secretly a Reward Model

3. Preference Probabilities from Policies

$$P(y_w \succ y_l \mid x) = \sigma(R(x, y_w) - R(x, y_l))$$

$$R(x, y_w) - R(x, y_l) = \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} + \underbrace{\beta \log Z(x) - \beta \log Z(x)}_{0}.$$

REPLACE π* WITH A TRAINABLE POLICY πθ

$$P_\theta(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

4. The Full DPO Loss

Each training pair identifies a winner. Maximize the probability of that observed preference.

$$\mathcal{L}_{\mathrm{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\mathrm{pref}}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\right)\right].$$

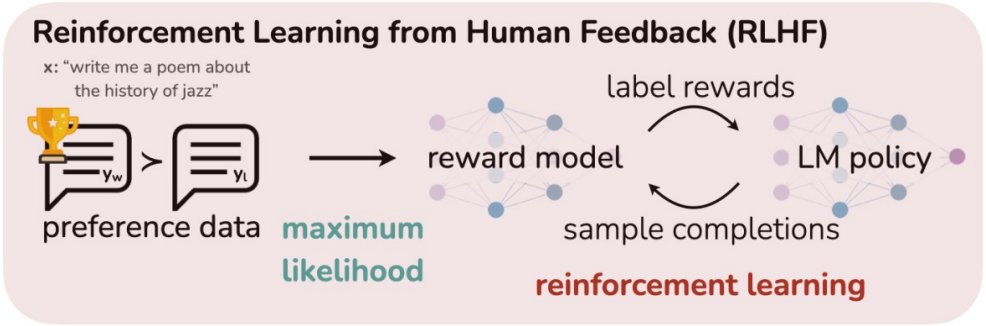
$$\log\pi_\theta(y \mid x) = \Sigma_{t=1}^{T}\log\pi_\theta(y_t \mid x, y_{<t})$$

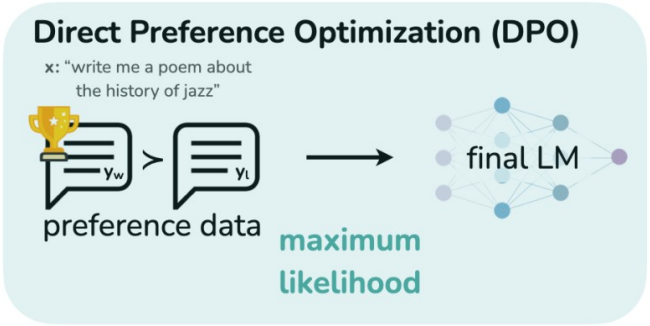
Four response log-probabilities. Actor trained, reference fixed. No separate reward model.



Rafailov et al., NeurIPS 2023 · Direct Preference Optimization: Your Language Model is Secretly a Reward Model

A Fixed-Dataset DPO Training Pipeline





DATA

fixed preference pairs

COMPUTE

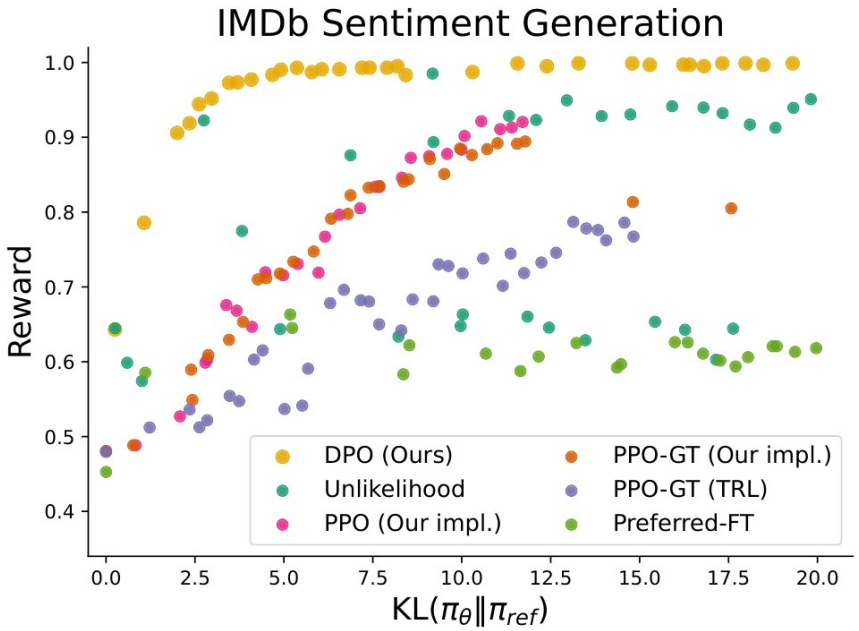
actor + reference log-probs

UPDATE

binary preference likelihood

The basic DPO update needs neither rollout generation nor a value critic.

DPO Can Optimize the Reward–KL Trade-Off



Controlled text generation

GPT-2-large completes IMDb reviews. A sentiment classifier supplies the target reward.

Read the frontier

x: KL to the reference.
y: mean sentiment reward.
Higher at the same KL is better.

One task and recipe; this does not establish a universal ranking.

Offline DPO and Online PPO

Training choice	Offline DPO	PPO with a reward model
Responses	Fixed chosen/rejected pairs	Fresh responses from the actor
Signal	Pair preference likelihood	Scores on the actor’s own outputs
Value model	No critic	V-model estimates advantages
Main cost	Training on saved pairs	Generation, scoring, actor + critic

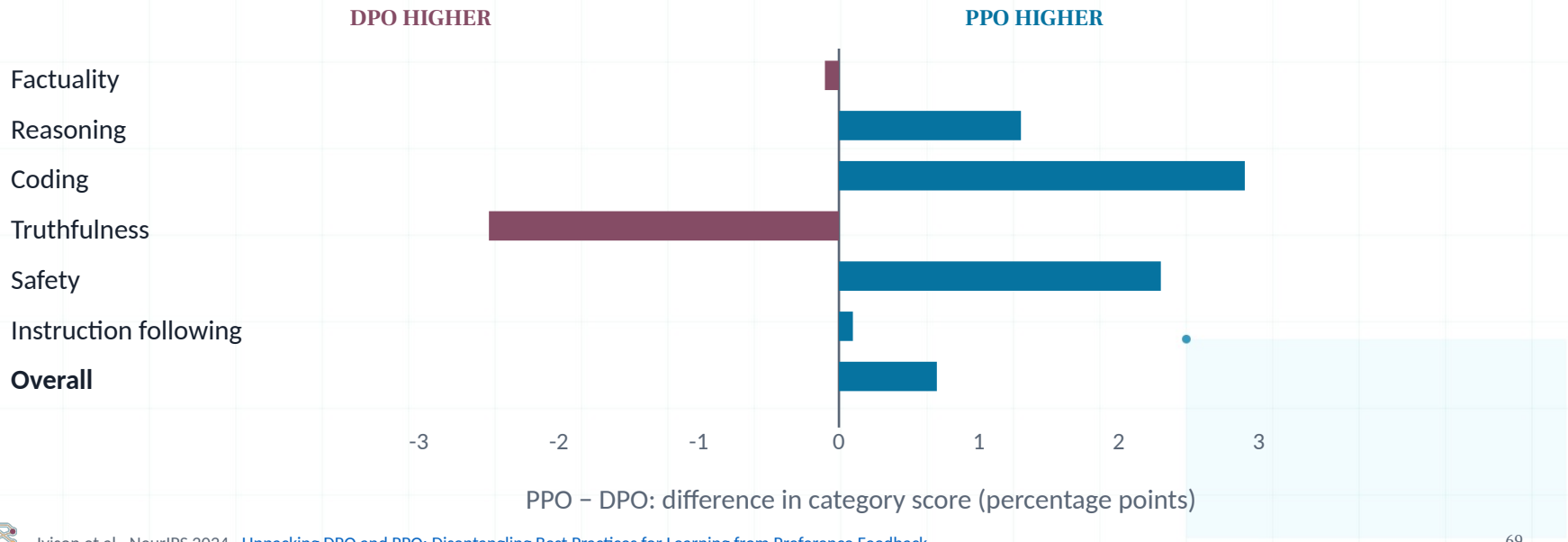
Online feedback can follow the changing policy. That advantage comes with additional compute.



Ivison et al., NeurIPS 2024 · Unpacking DPO and PPO: Disentangling Best Practices for Learning from Preference Feedback
Tang et al., 2024 · Understanding the Performance Gap between Online and Offline Alignment Algorithms

Compare the Whole Preference-Learning Recipe

TULU 2 · 13B | Same SFT start and preference data; compute is not matched.



Practical Limits of Offline DPO

Limitation	Consequence	What to monitor
Fixed pairs	New actor errors receive no new labels	Coverage of current responses
Preference labels	Noise and shortcuts shape learning	Label quality and task relevance
Implicit reward	Pair fit may miss generation quality	Held-out generated answers
Reference and β	No hard bound on the final KL	KL, response length, task scores

Online or iterative DPO can refresh pairs. The fixed-dataset limitation is a recipe choice.



Rafailov et al., NeurIPS 2023 · Direct Preference Optimization: Your Language Model is Secretly a Reward Model
Ivison et al., NeurIPS 2024 · Unpacking DPO and PPO: Disentangling Best Practices for Learning from Preference Feedback

08

What to Take Away

Exact identities, estimators, and algorithmic choices



Objective, Estimator, and Surrogate

Level	Examples	Question to ask
Objective	Expected return · reference KL	What counts as success?
Exact identities	Score function · baseline · Bellman	Which assumptions make it exact?
Estimators	REINFORCE · value targets · GAE	Where do noise and bias enter?
Surrogate / likelihood	PPO clipping · DPO preference loss	What is actually optimized?

Follow the chain from the desired behavior to the gradient actually computed.



Where This Lecture Sits in RL

RL

Policy optimization

REINFORCE → actor–critic → PPO

Value-based control

SARSA · Q-learning · DQN

Models and planning

known or learned dynamics → planning

ALONGSIDE: DPO · DIRECT PREFERENCE FITTING

On/off-policy and online/offline describe choices across these families.



Rafailov et al., NeurIPS 2023 · Direct Preference Optimization: Your Language Model is Secretly a Reward Model